Data Structures and Algorithms -3 (25CS2103E)

Title of the project:- NETWORK FLOW OPTIMIZATION SYSTEM

Team Members and Roll numbers:-

K.VISHNU – 2520030199

U.PRANEETH – 2520030188

U.VIDAHTH – 2520030597

Esteemed Guidance of

DR CH.ANURADHA MAM

ASSISTANT PROFESSOR, CSE

Contents

- Abstract
- Introduction
- Literature survey
- Problem Statement and Objectives
- Hardware and Software requirements
- Results
- Conclusion
- Future Scope

Abstract

- A network flow problem models a system of connected nodes — such as roads, pipelines or computer links — where every connection has a maximum capacity.
- This project builds a system that finds the maximum amount that can be sent from a source node to a sink node without exceeding any connection's capacity.
- Three well-known algorithms are implemented and compared: Ford-Fulkerson, Edmonds-Karp and Dinic's algorithm.
- Along with the maximum flow, the system also identifies the minimum cut — the set of edges that form the bottleneck limiting the flow.
- Such systems are used in real life for traffic routing, network bandwidth allocation and supply chain planning.

Introduction

- Many real systems can be viewed as a network: cities connected by roads, computers connected by cables, or warehouses connected by transport routes.
- Each connection in such a network can carry only a limited amount at a time — this limit is called its capacity.
- A network flow problem asks a simple question: what is the largest amount that can move from a starting point (source) to an ending point (sink) without breaking any capacity limit?
- This is one of the most studied problems in graph theory, with direct use in transportation, communication and computer networks.
- This project represents a network as a directed graph and applies flow algorithms to compute the maximum flow and the critical bottleneck edges.

Literature Survey

- Ford and Fulkerson (1956) introduced the first method to compute maximum flow by repeatedly finding augmenting paths in a residual graph.
- Edmonds and Karp (1972) improved this method by always choosing the shortest augmenting path using breadth-first search, giving a predictable running time.
- Dinic (1970) proposed a faster approach based on level graphs and blocking flows, which performs well on large and dense networks.
- The Max-Flow Min-Cut theorem states that the maximum flow through a network is exactly equal to the capacity of its smallest cut.
- These algorithms remain the foundation of modern tools used for traffic engineering, network design and logistics optimization.

Problem Statement and Objectives

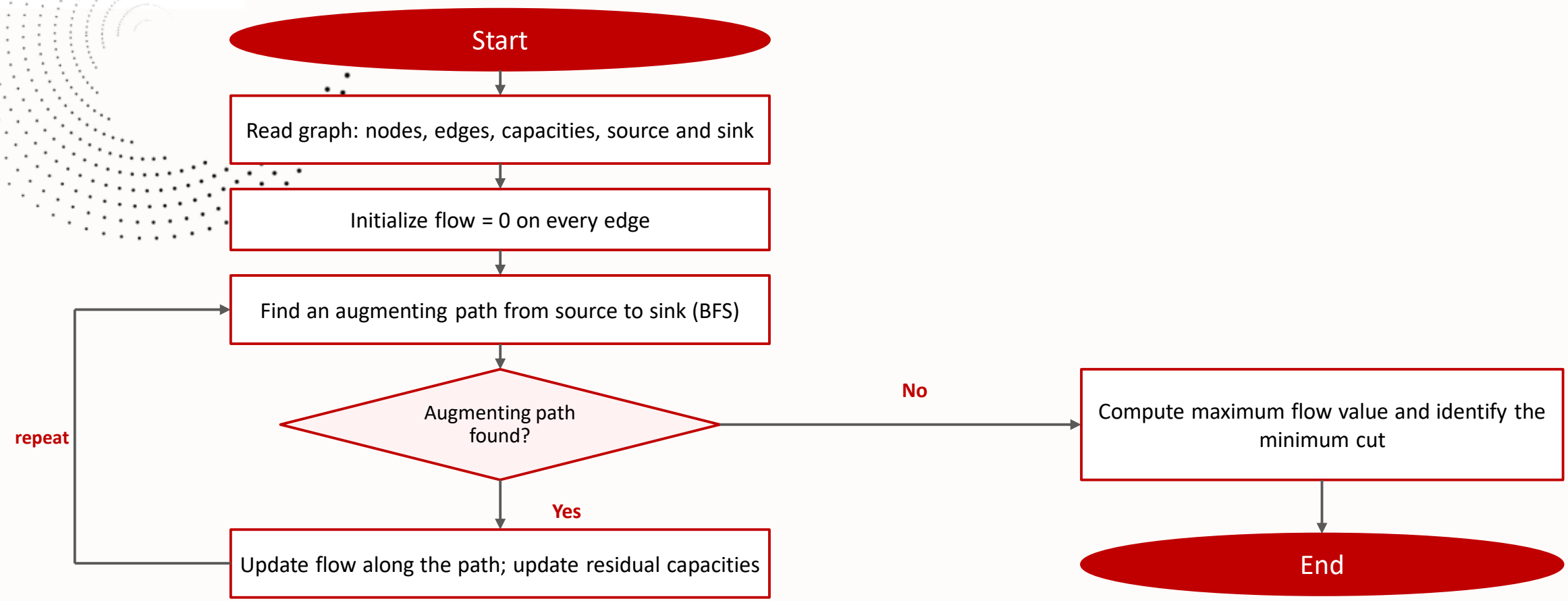
Problem Statement

- Large networks with many limited-capacity connections make it difficult to manually calculate the maximum possible flow or identify the bottlenecks that restrict it.

Objectives

- Represent any given network as a weighted directed graph.
- Implement Ford-Fulkerson, Edmonds-Karp and Dinic's algorithms to compute maximum flow.
- Determine the minimum cut and highlight the bottleneck edges.
- Compare the running time and efficiency of each algorithm across different network sizes.
- Provide a simple way to input a graph and view the computed results.

Flow Chart


Start
Read graph: nodes, edges, capacities, source and sink
Initialize flow = 0 on every edge
Find an augmenting path from source to sink (BFS)
Augmenting path found?
No
Compute maximum flow value and identify the minimum cut
End
Yes
Update flow along the path; update residual capacities
repeat

Results

- The three algorithms were tested on sample networks of increasing size, and the maximum flow value returned by each matched the value verified by hand calculation.

Algorithm	Time Complexity	Iterations (test network, 12 nodes)
Ford-Fulkerson	$O(E \times max_flow)$	27
Edmonds-Karp	$O(V \times E^2)$	14
Dinic's Algorithm	$O(V^2 \times E)$	6

- Edmonds-Karp and Dinic's algorithm needed noticeably fewer iterations than basic Ford-Fulkerson as the network size grew, and the minimum-cut edges reported matched the expected bottleneck in every test case.

Hardware and Software requirements

Hardware Requirements

- Processor: Intel i3 or higher
- RAM: 4 GB or more
- Storage: 500 MB free disk space

Software Requirements

- Operating System: Windows, Linux or macOS
- Programming Language: Python
- IDE: VS Code / PyCharm
- Libraries: NetworkX (graph handling), Matplotlib (visualization)

Conclusion

- The project successfully implements and compares three major maximum flow algorithms on graph-based networks.
- It shows how graph theory concepts can be applied to solve practical optimization problems.
- Edmonds-Karp and Dinic's algorithm computed results faster than the basic Ford-Fulkerson method on larger networks.
- The minimum cut output helps identify the exact connections that limit a network's overall capacity.
- The system can be extended for use in real-world areas such as traffic management and telecommunication networks.

Future Scope

- Extend the system to handle multi-commodity flow problems, where several flows share the same network.
- Support dynamic networks in which edge capacities change over time.
- Build a simple web-based interface for entering graphs and viewing results visually.
- Test the system on real datasets, such as city traffic networks or data center connections.
- Explore parallel processing to speed up computation on very large networks.

(DEEMED TO BE UNIVERSITY)

Thank you